Python - Modify Strings

String modification refers to the process of changing the characters of a string. If we talk about modifying a string in Python, what we are talking about is creating a new string that is a variation of the original one.

In Python, a string (object of **str** class) is of immutable type. Here, immutable refers to an object thatcannotbe modified in place once it's created in memory. Unlike a list, we cannot overwrite any character in the sequence, nor can we insert or append characters to it directly. If we need to modify a string, we will use certain string methods that return anewstring object. However, the original string remains unchanged.

We can use any of the following tricks as a workaround to modify a string.

Converting a String to a List

Both strings and lists in Python are sequence types, they are interconvertible. Thus, we can cast a string to a list, modify the list using methods like insert(), append(), or remove() and then convert the list back to a string to obtain a modified version.

Suppose, we have a string variable **`s1`** with **`WORD`** as its value and we are required to convert it into a list. For this operation, we can use the list() built-in function and insert a character **`L`** at index 3. Then, we can concatenate all the characters using join() method of str class.

Example

The below example practically illustrates how to convert a string into a list.

```
s1="WORD"
print ("original string:", s1)
l1=list(s1)
```

```
l1.insert(3,"L")

print (l1)

s1=''.join(l1)
print ("Modified string:", s1)
```

It will produce the following **output** −

```
original string: WORD
['W', 'O', 'R', 'L', 'D']
Modified string: WORLD
```

Using the Array Module

To modify a string, construct an array object using the Python standard library named **`array module`**. It will create an array of Unicode type from a string variable.

Example

In the below example, we are using array module to modify the specified string.

```
import array as ar

# initializing a string
s1="WORD"
print ("original string:", s1)

# converting it to an array
sar=ar.array('u', s1)

# inserting an element
sar.insert(3,"L")

# getting back the modified string
s1=sar.tounicode()
print ("Modified string:", s1)
```

It will produce the following **output** –

```
original string: WORD
Modified string: WORLD
```

Using the StringIO Class

Python's io module defines the classes to handle streams. The StringIO class represents a text stream using an in-memory text buffer. A StringIO object obtained from a string behaves like a File object. Hence we can perform read/write operations on it. The getvalue() method of StringIO class returns a string.

Example

Let us use the above discussed principle in the following program to modify a string.

```python
import io

s1="WORD"
print ("original string:", s1)

sio=io.StringIO(s1)
sio.seek(3)
sio.write("LD")
s1=sio.getvalue()

print ("Modified string:", s1)
```

It will produce the following **output** –

```
original string: WORD
Modified string: WORLD
```

Java Tutorial
C++ Tutorial
C Programming Tutorial
C# Tutorial
PHP Tutorial
R Tutorial
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

Chapters

Categories

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler

Online Python Compiler

Online Go Compiler

Online C Compiler

Online C++ Compiler

Online C# Compiler

Online PHP Compiler

Online MATLAB Compiler

Online Bash Terminal

Online SQL Compiler

Online Html Editor